

STATISTICAL ARBITRAGE · บทที่ 23 — บทเสริม ADVANCED

Kalman Filter & t-Distribution

Dynamic β_t | Predict-Update Cycle | Fat Tails | Critical Values

สัญญาณซื้อขายได้ Normal vs t-Distribution

แก่นของบทนี้ — อ่านก่อน

Kalman Filter แก้ปัญหา β_t ที่เปลี่ยนตามเวลา — ปรับ β ทุก tick แทน rolling window

t-Distribution แก้ปัญหา fat tails ของ ε crypto — threshold ที่ถูกต้องกว่า z-score ปกติ
ทั้งสองเทคนิคเสริมกัน: Kalman ให้ β_t ที่แม่นยำ → t-distribution แปลง ε เป็น signal ที่น่าเชื่อถือกว่า

$$z_t = \varepsilon_t / \sigma_t \rightarrow \text{เทียบกับ } t_{\{\alpha/2\}}(v) \text{ ไม่ใช่ } z_{\{\alpha/2\}} = 1.96$$

Prerequisites — ต้องเข้าใจก่อนอ่านบทนี้

- [บทที่ 3](#) §3.3 Half-life และ AR(1) ของ ε
- [บทที่ 4](#) §4.2 OLS สำหรับ β (static β foundation)
- [บทที่ 5](#) §5.6 Shapiro-Wilk, §5.7 Crossing Statistics
- [บทที่ 10](#) §10.3 Robust Z-score (MAD-based)
- [บทที่ 22](#) §22.3 Risk Reversal, §22.5 VRP (ε ที่มี fat tails)

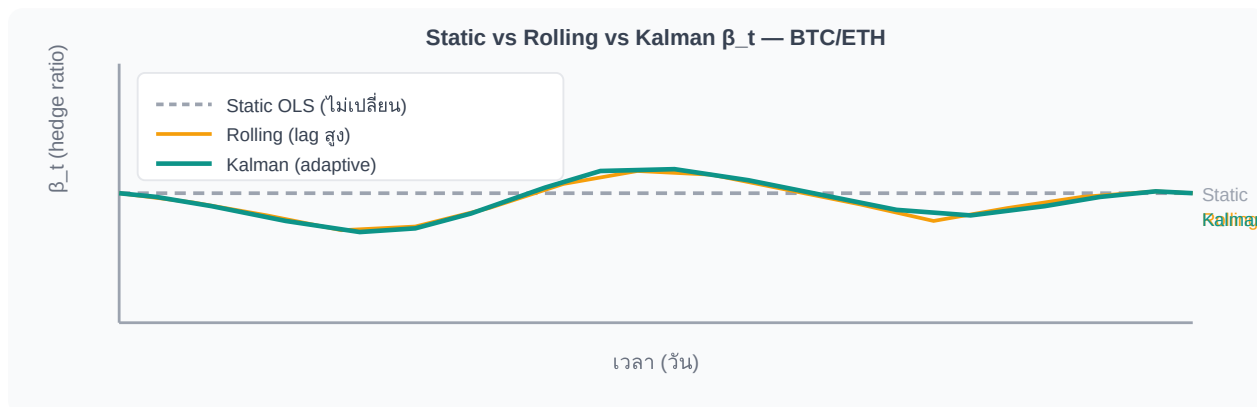
23.1 ปัญหาของ β คงที่ — ทำไม OLS ไม่พอ

ใน [บทที่ 4](#) เราประมาณ β ด้วย OLS บน lookback window เดียว เช่น 250 วัน โมเดลถือว่า β ไม่เปลี่ยนแปลงตลอดช่วงเวลานั้น แต่ในทางปฏิบัติ hedge ratio ของ crypto pairs เช่น BTC/ETH ขยับตามรอบตลาด

สามข้อจำกัดของ Static OLS β

1. **Window selection dilemma:** Window สั้น (30 วัน) → β ตอบสนองเร็วแต่ noisy มาก | Window ยาว (250 วัน) → smooth แต่ lag สูง lag เกิดขึ้นเมื่อรอบตลาดเปลี่ยน
2. **Regime change blindness:** เมื่อ correlation ระหว่าง BTC/ETH ลดลงชั่วคราว (เช่น altcoin season) OLS β ที่ fit บน data เก่าจะให้ ϵ ที่ biased — z-score ผิดพลาด ทำให้เข้า trade ผิด side
3. **Abrupt structural break:** Hard fork, exchange hack, macro event → β เปลี่ยนทันที แต่ OLS ใช้เวลาหลายสัปดาห์กว่าจะ update — ช่วงนั้น strategy ขาดทุนทุก trade

Rolling-window β เป็นทางออกแรก: คำนวณ β ใหม่ทุกวันบน lookback window ที่เลื่อนไป แต่ยังเลือก window ได้ยาก และยังมี lag



รูป 23.1 — Kalman β_t (เขียว) ตอบสนองรวดเร็วกว่า rolling window (เหลือง) โดยไม่มี lag สูง

Kalman Filter แก้ทั้งสามปัญหาในคราวเดียว: ปรับ β ทุก step, ไม่ต้องเลือก window, และ update ทันทีเมื่อ data ใหม่เข้า

23.2 Kalman Filter — ทฤษฎีและวงจร Predict-Update

Kalman Filter คือ **recursive Bayesian estimator** — ในแต่ละ step เราประมาณ state ล่าสุดจากการผสม prediction จาก model กับ observation ใหม่ น้ำหนักที่ให้แต่ละส่วนขึ้นอยู่กับความไม่แน่นอนสัมพัทธ์ของทั้งสอง

State-Space Model สำหรับ β_t

กำหนด state equation และ observation equation:

```
// State equation ( $\beta$  เปลี่ยนช้าๆ แบบ random walk)

 $\beta_t = \beta_{t-1} + w_t, \quad w_t \sim N(0, Q) \quad // Q = \text{process noise variance}$ 

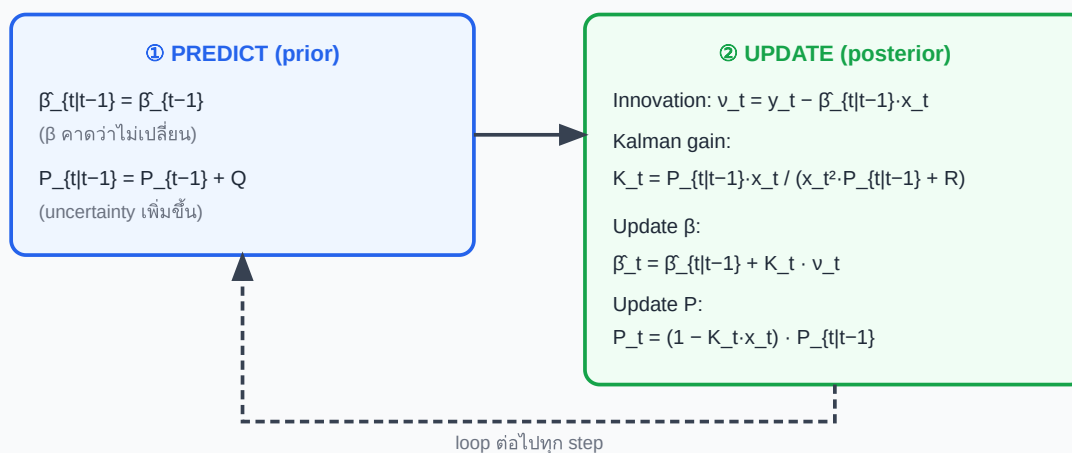
// Observation equation (ราคาที่เราสังเกตได้)

 $y_t = \beta_t \cdot x_t + v_t, \quad v_t \sim N(0, R) \quad // R = \text{observation noise variance}$ 
```

- **Q (Process noise):** ควบคุมความเร็วที่ β_t เปลี่ยนได้ — ค่ามากขึ้น $\rightarrow \beta$ ตอบสนองเร็วขึ้นแต่ noisy มากขึ้น
- **R (Observation noise):** สะท้อน noise ใน price relationship — ค่ามากขึ้น $\rightarrow$ filter เชื่อ observation น้อยลง เชื่อ model มากขึ้น
- **P_t (Error covariance):** ความไม่แน่นอนของ estimate β_t — ยิ่งสูง filter ยิ่งให้น้ำหนัก observation ใหม่มากขึ้น

วงจร Predict-Update ทุก time step

Kalman Filter — Predict-Update Cycle



รูป 23.2 — วงจร Predict-Update ของ Kalman Filter ทำซ้ำทุก time step

ตีความ Kalman Gain K_t

K_t อยู่ใน range ที่เหมาะสม (สำหรับ log-return inputs ที่ $x_t \ll 1$):

- $K_t \rightarrow 1$: เชื่อ observation ใหม่เกือบทั้งหมด (P สูง หรือ R ต่ำ) $\rightarrow \beta$ ปรับตาม data ใหม่มาก
- $K_t \rightarrow 0$: เชื่อ prediction เกือบทั้งหมด (P ต่ำ หรือ R สูง) $\rightarrow \beta$ เปลี่ยนน้อยมาก
- $K_t \approx 0.5$: ถ่วงน้ำหนักเท่ากัน — filter อยู่ในสมดุล

23.3 Kalman Filter สำหรับ Dynamic β_t — BTC/ETH

นำทฤษฎีในข้อ §23.2 มาใช้กับ BTC/ETH pair จาก [บทที่ 4 §4.2b](#) — BTC เป็น y_t (dependent), ETH เป็น x_t (independent)

ตัวอย่างหลัก — BTC/ETH Dynamic β_t (log-returns)

Pair: BTC/ETH จาก [บทที่ 4 §4.2b](#) | $\rho = 0.9745$ | AR(1) $\phi = 0.9602$ | Half-life = 17h

Input scale: ใช้ *log-returns* ($r_t = \Delta \log P_t$) ไม่ใช่ *log-prices* — ทำให้ innovation มี mean ≈ 0 โดยไม่ต้อง fit intercept และ K_t อยู่ใน range ที่ guard condition ทำงานได้

Log-return OLS β : $\beta_0 \approx 1.10$ (BTC return $\approx 1.10 \times$ ETH return) — แตกต่างจาก $\beta=0.0444$ ซึ่งเป็น raw-price slope; ดู §4.2b

Kalman parameters: $Q = 1 \times 10^{-5}$ (β เปลี่ยนช้า), $R = 0.001$ (noise ปานกลาง)

Initial conditions: $\beta_0 = 1.10$ (ค่าจาก OLS log-returns), $P_0 = 0.01$

Pseudo-code: Kalman Filter สำหรับ β_t

```
# ตั้งค่าเริ่มต้น
beta_hat = 1.10 # initial  $\beta$  (จาก OLS log-returns BTC/ETH)
P = 0.01 # initial error covariance
Q = 1e-5 # process noise (tuning parameter)
R = 0.001 # observation noise (tuning parameter)
beta_series = []

for t in range(len(prices)):
    y_t = r_btc[t]
    # log-return  $\Delta \log(P_{BTC_t})$ 
    x_t = r_eth[t]
    # log-return  $\Delta \log(P_{ETH_t})$ 

    ## ① PREDICT
    beta_prior = beta_hat #  $\beta_{t|t-1}$ 
    P_prior = P + Q #  $P_{t|t-1}$ 

    ## ② UPDATE
```

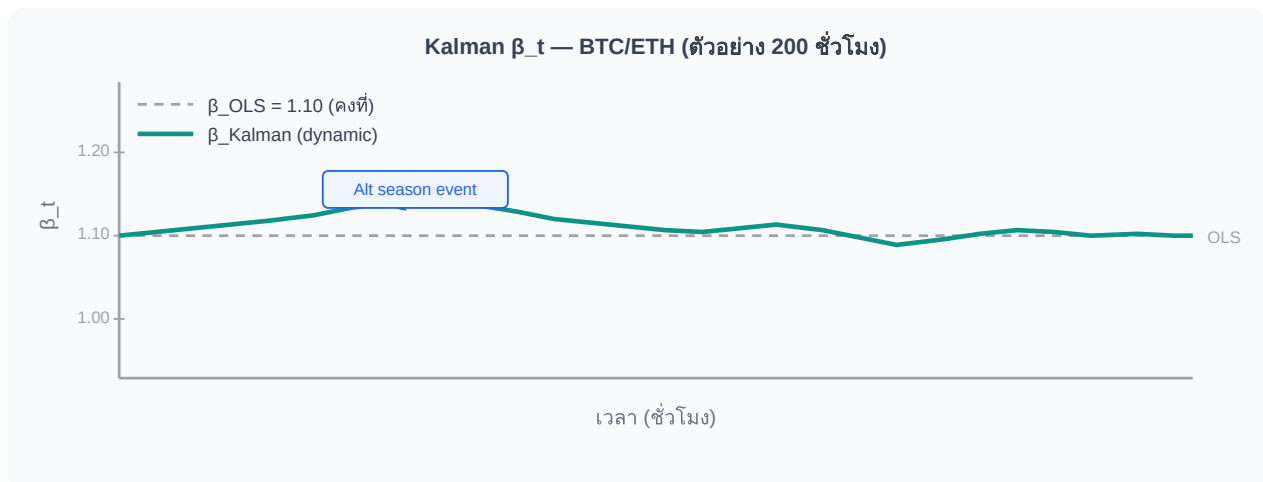
```

innovation = y_t - beta_prior * x_t # v_t (residual)
S = x_t**2 * P_prior + R # innovation variance
K = P_prior * x_t / S # Kalman gain
beta_hat = beta_prior + K * innovation
P = (1 - K * x_t) * P_prior

## คำนวณ  $\epsilon_t$  ด้วย  $\beta_t$  ที่ update แล้ว
epsilon_t = y_t - beta_hat * x_t
beta_series.append(beta_hat)

```

หลังจากวน loop เสร็จ `beta_series` คือ time series ของ β_t ที่ปรับตาม data ทุก step โดยอัตโนมัติ สำหรับ BTC/ETH log-returns β_t โคจรรอบ $\sim 1.10 \pm 0.1$ ตามรอบตลาด



รูป 23.3 — β_t ของ Kalman Filter (เขียว) เลื่อนขึ้นระหว่าง altcoin season แล้วกลับสู่ระดับปกติ ขณะที่ OLS คงที่ตลอด (log-return scale: $\beta \approx 1.0\text{--}1.2$)

ข้อดีเชิงปฏิบัติ — ϵ_t ที่แม่นยำกว่า

เมื่อใช้ Kalman β_t แทน OLS β :

- $\epsilon_t = y_t - \beta_t \cdot x_t$ มี **variance ต่ำกว่า** และ **mean ≈ 0** สม่่าเสมอกว่า
- z-score threshold ยังใช้เดิม ($\pm 2\sigma$) แต่ **false signals ลดลง** เพราะ ϵ ไม่มี drift จาก stale β
- Half-life ของ ϵ_t สั้นลง $\rightarrow$ reversion เร็วขึ้น $\rightarrow$ สามารถ trade ได้ถี่ขึ้นหรือ hold time ลดลง

23.4 ข้อจำกัดและการ Tune Kalman Filter

Kalman Filter มีพลัง แต่ต้องเข้าใจข้อจำกัดก่อนนำไปใช้จริง

⚠ ข้อจำกัดสำคัญ 4 ข้อ

- Q/R tuning:** ไม่มีสูตรตายตัวสำหรับ Q และ R — ต้องทำ walk-forward validation (ch3 §3.9) เพื่อหาค่าที่ให้ Sharpe ratio สูงสุดบน out-of-sample data
- Initial conditions sensitivity:** ค่าเริ่มต้น β_0 และ P_0 ส่งผลต่อ 50–100 step แรก — burn-in period นี้ควรตัดออกจาก live trading ค่อยๆ converge
- Linearity assumption:** Model นี้สมมติ β เปลี่ยนแบบ linear random walk (Gaussian) — ถ้า jump discontinuity เกิดขึ้น (เช่น delisting) Kalman จะ overreact หรือ underreact
- Single-factor limitation:** Filter ข้างบน estimate β เดียว (slope) โดยถือ intercept ≈ 0 สำหรับ model ที่มีทั้ง α และ β ต้องขยายเป็น state vector 2×1 (Kalman เวอร์ชัน multivariate)
- Numerical stability:** $P_t = (1 - K_t \cdot x_t) \cdot P_{\text{prior}}$ อาจมีค่าติดลบจาก floating-point rounding ในระยะยาว — production code ควรใช้ Joseph form: $P_t = (1 - K_t \cdot x_t) \cdot P_{\text{prior}} \cdot (1 - K_t \cdot x_t) + K_t^2 \cdot R$ เพื่อรับประกัน $P_t > 0$ เสมอ

แนวทาง Tune Q และ R

Parameter	ค่าต่ำ	ค่าสูง	Rule of thumb
Q (process noise)	β เปลี่ยนช้า — smooth แต่ lag	β ตอบสนองเร็ว — แต่ noisy	เริ่มที่ $1e-5$ ถึง $1e-4$; หาจาก grid search
R (observation noise)	เชื่อ price มาก — β วิ่งตาม	เชื่อ model มาก — β stable	ประมาณจาก $\text{Var}(\epsilon)$ ของ OLS residual
P_0 (initial P)	confident เรื่อง β_0	uncertain เรื่อง β_0	ถ้าไม่แน่ใจ ใส่ $P_0 = 1.0$ แล้ว burn-in

EM Algorithm สำหรับ Q/R (Advanced)

สามารถประมาณ Q และ R จาก data โดยตรงด้วย **Expectation-Maximization (EM)** บน log-likelihood ของ state-space model — ไม่ต้อง tune ด้วยมือ แต่ต้องการ training window ยาวพอ

(อย่างน้อย 500+ observations) ห้องสมุด Python ที่ใช้ได้: `pykalman`,
`statsmodels.tsa.statespace`

เส้นทางจาก §23.1–23.4 ครอบคลุม Dynamic β ด้วย Kalman Filter ครบถ้วน ส่วนต่อไป §23.5–23.8 จะแก้
อีกปัญหาหนึ่งที่เกี่ยวข้องกัน — fat tails ของ ϵ ที่ทำให้ z-score แบบ Normal ไม่น่าเชื่อถือ

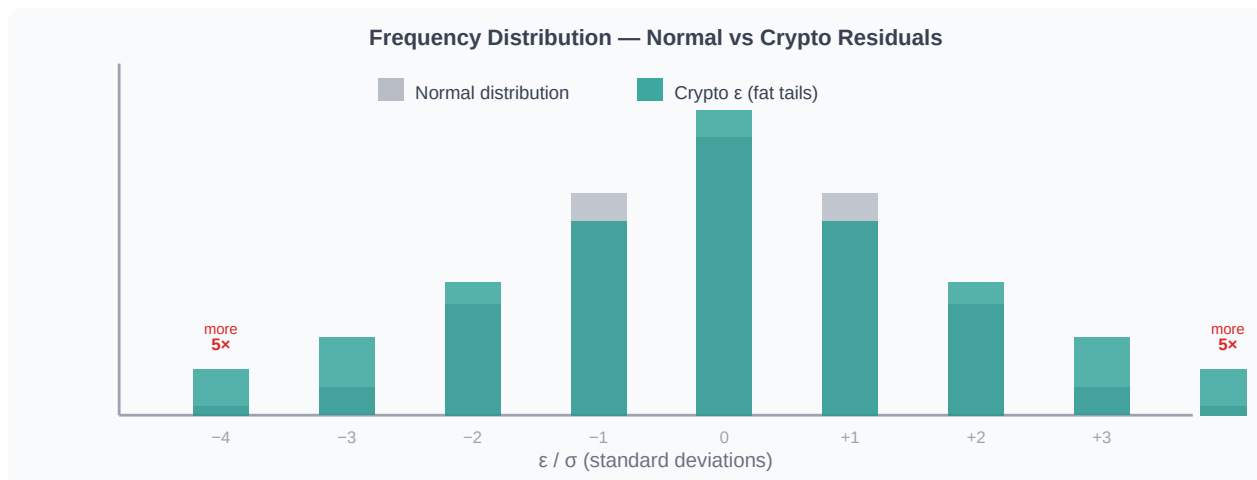
23.5 ปัญหา Fat Tails ใน Crypto Residuals

ϵ ของ crypto pairs และ options residuals (เช่น ϵ_{RR} จาก §22.3) มักมีหางที่หนากว่าการแจกแจงปกติมาก — ปัญหานี้ทำให้ z-score ที่คำนวณจาก Normal distribution ให้ false signals มากเกินจริง

Kurtosis — วัดความ "heavy" ของหาง

$$\text{Kurtosis (excess)} = E[(\epsilon/\sigma)^4] - 3$$

- **Normal distribution:** excess kurtosis = 0 (kurtosis = 3)
- **BTC/ETH residuals:** excess kurtosis $\approx 3-9$ (kurtosis 6–12) — เหตุการณ์ 4 σ เกิดถี่กว่า Normal หลายเท่า
- **ϵ_{RR} (Risk Reversal):** excess kurtosis $\approx 5-12$ — IV spike ระหว่าง fear events สร้าง extreme ϵ



รูป 23.4 — ที่ $\pm 4\sigma$ crypto residuals เกิดถึง 5× บ่อยกว่า Normal — z-score 2.0 แบบปกติให้ false entries มากเกิน

ผลต่อ Trading Signals

ถ้าใช้ z-score threshold = ± 2.0 โดยสมมติ Normal distribution:

- ตาม Normal: $P(|z| > 2.0) = 4.6\%$ → คาด 4.6% ของ observations เป็น signal
- สำหรับ crypto ϵ ที่มี excess kurtosis = 6: $P(|z| > 2.0) \approx 8-12\%$ จริงๆ → **signals เกือบ 2–3× มากเกินที่ควรจะเป็น**
- ส่วนใหญ่ที่เกินมาคือ **false signals** จากหาง → เข้า trade ตอน momentum ยังแรง แล้วขาดทุน

ทางออก: ใช้ **t-distribution** ที่มี **degrees of freedom v** ต่ำ เพื่อสะท้อน fat tails ที่แท้จริง

23.6 t-Distribution — สูตรและ Degrees of Freedom

t-distribution (Student's t) มีรูปร่างคล้าย Normal แต่มีหางหนักกว่าที่ปรับได้ด้วยพารามิเตอร์ **degrees of freedom ν (nu)**

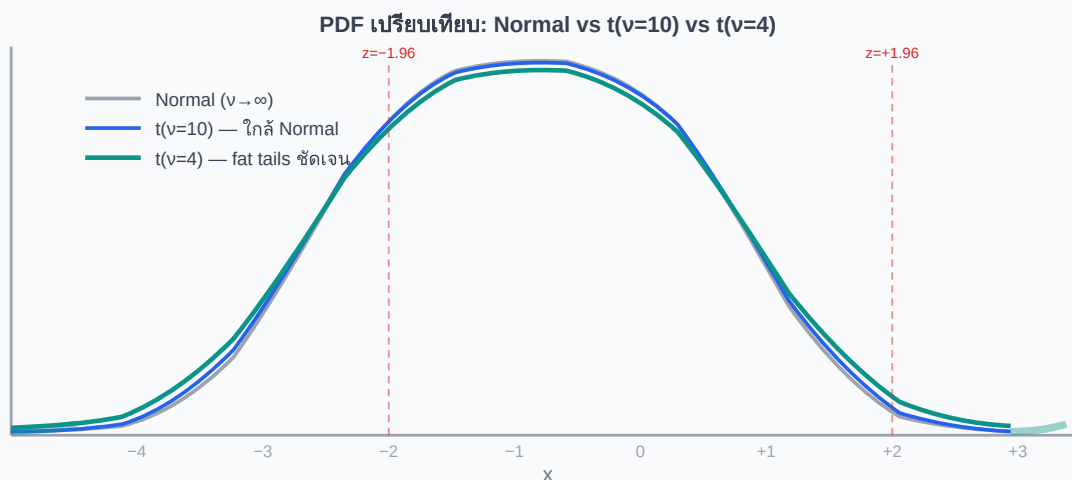
Probability Density Function

$$f(t; \nu) = \frac{\Gamma((\nu+1)/2)}{(\sqrt{\nu\pi}) \cdot \Gamma(\nu/2)} \cdot (1 + t^2/\nu)^{-(\nu+1)/2}$$

เมื่อ $\nu \rightarrow \infty$: $f(t;\nu) \rightarrow \text{Normal PDF } N(0,1)$

เมื่อ $\nu \downarrow 1$: หางหนักมาก (Cauchy distribution)

สำหรับ crypto: $\nu \approx 3-6$ ให้ผลที่ fit data ดีที่สุด



รูป 23.5 — $t(\nu=4)$ มีหางหนักกว่า Normal ชัดเจน — area นอก ± 1.96 ใหญ่กว่า $2\times$ สำหรับ crypto $\nu \approx 3-6$

คุณสมบัติสำคัญของ t-distribution

ν (degrees of freedom)	ลักษณะ	Excess Kurtosis	Variance
$\nu = 2$	หางหนักมาก	ไม่มีที่สิ้นสุด	ไม่มีที่สิ้นสุด
$\nu = 3$	หางหนา	ไม่มีที่สิ้นสุด	3.0

ν (degrees of freedom)	ลักษณะ	Excess Kurtosis	Variance
$\nu = 4$	fat tails — โมเมนต์ที่ 4 ไม่มีที่สิ้นสุด	ไม่มีที่สิ้นสุด	2.0
$\nu = 5$	fat tails — เหมาะ crypto มาก	6.0	1.67
$\nu = 6$	moderate fat tails	3.0	1.5
$\nu = 10$	ใกล้ Normal	1.0	1.25
$\nu \rightarrow \infty$	Normal distribution	0	1.0

ตัวอย่างหลัก — ε_{RR} (Risk Reversal) Options Residual

ε_{RR} จาก §22.3 ของ BTC options มี distribution ที่ fat-tailed เนื่องจาก IV spikes ระหว่าง market stress:

- BTC 30-day IV สามารถ jump จาก 50% $\rightarrow$ 120% ภายใน 48 ชั่วโมง (Black Thursday 2020, FTX collapse 2022)
- MLE estimate $\nu \approx 4-5$ สำหรับ ε_{RR} บน 1-year data
- ถ้าใช้ Normal threshold $z = 2.0 \rightarrow$ จะ flag ε_{RR} ที่ IV spike ว่าเป็น "entry signal" ทั้งที่จริงๆ คือ regime change ที่ควรหลีกเลี่ยง
- $t(\nu=4)$ threshold = 2.78 $\rightarrow$ กรองสัญญาณ IV spike ออกไปได้ดีกว่า

23.7 Critical Values — Normal vs t-Distribution

ความแตกต่างใน threshold นี้มีผลโดยตรงต่อจำนวนครั้งที่ strategy เข้า trade

Distribution	Two-tailed $\alpha=5\%$	Two-tailed $\alpha=2\%$	Two-tailed $\alpha=1\%$
Normal (z)	1.960	2.326	2.576
t (v = 30)	2.042	2.457	2.750
t (v = 10)	2.228	2.764	3.169
t (v = 6)	2.447	3.143	3.707
t (v = 4)	2.776	3.747	4.604
t (v = 5)	2.571	3.365	4.032
t (v = 3)	3.182	4.541	5.841

ตีความตาราง

ถ้าใช้ $t(v=4)$ แทน Normal สำหรับ two-tailed $\alpha=5\%$ (threshold แบบ "95% confidence"):

- Normal threshold: $|z| \geq 1.96$
- $t(v=4)$ threshold: $|t| \geq 2.78 \rightarrow$ เข้มกว่า 42%
- ความหมาย: ϵ ต้องห่างจาก mean มากกว่า 42% จึงจะเรียกว่า "significant" — กรอง noise ออกมากขึ้น

การประมาณ v จากข้อมูล (MLE)

หา v ที่เหมาะสมกับ ϵ โดยใช้ Maximum Likelihood Estimation:

```
from scipy.stats import t as t_dist
import numpy as np

# MLE fit ของ t-distribution บน  $\epsilon_{RR}$  หรือ  $\epsilon$  BTC/ETH
epsilon = np.array([...]) # residuals series
```

```
# fit returns (df, loc, scale) – df คือ v ที่ fit ดีที่สุด
nu, loc, scale = t_dist.fit(epsilon, floc=0) # fix loc=0 (mean-zero  $\epsilon$ )

print(f"Estimated v = {nu:.2f}") # ตัวอย่าง: v ≈ 4.2

# critical value สำหรับ two-tailed 5%
threshold = t_dist.ppf(0.975, df=nu) # ตัวอย่าง: 2.74
print(f"Use threshold ±{threshold:.3f} instead of ±1.960")
```

ตัวอย่างหลัก — MLE บน ϵ_{RR} BTC Options

Input: ϵ_{RR} series จาก 1 ปีของ BTC 25 Δ Risk Reversal (≈ 250 daily observations)

MLE result: $\hat{v} \approx 4.3 \rightarrow$ ยืนยันว่า fat-tailed

New threshold: $t_{\{0.025\}}(v=4.3) \approx 2.72$ (แทน 1.96)

ผลต่อ strategy:

- Signals ลดลงจาก ≈ 28 ครั้ง/ปี (Normal) $\rightarrow \approx 17$ ครั้ง/ปี (t-dist)
- False positive rate ลดลงจาก $\approx 8\%$ $\rightarrow \approx 4.8\%$ (ใกล้เคียงทางทฤษฎี $\alpha=5\%$ จริงๆ)
- Win rate ต่อ trade สูงขึ้น แต่ trade น้อยลง — net Sharpe ratio ดีขึ้น

ตารางและตัวอย่างข้างต้นแสดงว่า threshold ที่ถูกต้องสำหรับ crypto ϵ ต้องสูงกว่า 1.96 อย่างมีนัยสำคัญ ส่วน §23.8 จะแสดงผลกระทบนี้ต่อ trading signals จริงว่าต่างกันมากแค่ไหนในทางปฏิบัติ

23.8 เปรียบเทียบ Trading Signals: Normal vs t-Distribution

ส่วนนี้แสดงผลต่างเชิงปฏิบัติของการใช้ Normal vs t-distribution ในการสร้าง trading signals บน pipeline เดิม

Scenario	Normal (z=2.0)	t(v=4, t=2.78)	ผล
$\epsilon_{RR} = -12\%$ ($\sigma_{RR}=2\%$, $\theta=-8\%$)	$z = (-12-(-8))/2 = -2.0 \rightarrow$ ENTER	$t = -2.0 < 2.78 \rightarrow$ WAIT	t กรอง marginal signal
$\epsilon_{RR} = -15\%$ ($\sigma_{RR}=2\%$, $\theta=-8\%$)	$z = -3.5 \rightarrow$ ENTER	$t = -3.5 > 2.78 \rightarrow$ ENTER	ทั้งคู่เห็นตรงกัน
$\epsilon_{BTC} = +2.3\sigma$ (half-life event)	$z = 2.3 \rightarrow$ ENTER	$t = 2.3 < 2.78 \rightarrow$ WAIT	t กรอง noise trade
$\epsilon_{BTC} = +3.1\sigma$ (IV spike event)	$z = 3.1 \rightarrow$ ENTER	$t = 3.1 > 2.78 \rightarrow$ ENTER	ทั้งคู่เห็นตรงกัน
$\epsilon_{PCP} = -1.8\sigma$ (PCP violation)	$z = 1.8 \rightarrow$ WAIT	$t = 1.8 \rightarrow$ WAIT	ทั้งคู่ผ่าน — ถูกต้อง

สรุปผลต่างเชิงปฏิบัติ

- **t-distribution ช่วยลด false positives** โดยเฉพาะสัญญาณที่ z อยู่ในโซน 2.0–2.7 — ซึ่งใน crypto เป็น noise มากกว่า signal
- Signals ที่ผ่าน t-threshold มักอยู่ใกล้ขอบเขต extreme ของ distribution จริงๆ $\rightarrow$ higher conviction trades
- สำหรับ **options ϵ (ϵ_{RR} , ϵ_{PCP})**: ควรใช้ t เสมอเพราะ fat tails จาก IV dynamics รุนแรงกว่า spot ϵ
- สำหรับ **spot pair ϵ** ที่ผ่าน Shapiro-Wilk (ch5 §5.6): ใช้ Normal ก็ได้ — แต่ถ้า kurtosis $> 5 \rightarrow$ เปลี่ยนเป็น t

§23.5–23.8 แสดงว่า t-distribution ให้ threshold ที่สะท้อน tail risk ของ crypto จริงมากกว่า ส่วนต่อไป §23.9 จะรวม Kalman Filter กับ t-distribution เป็น pipeline เดียว

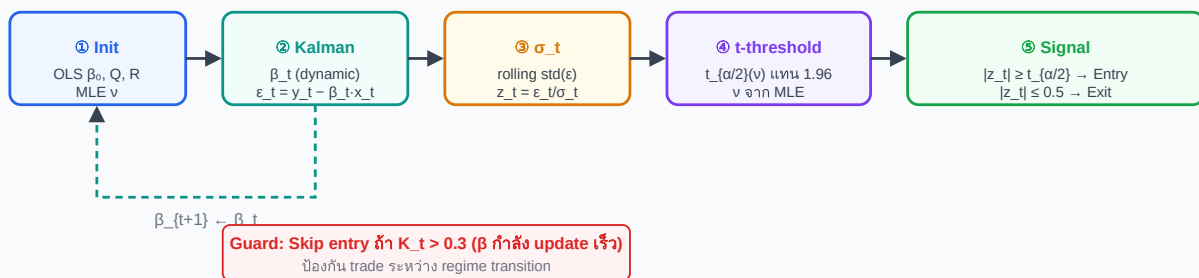
23.9 Pipeline รวม: Kalman β_t + t-Distribution Signal

Pipeline ต่อไปนี้รวม 2 เทคนิคจาก §23.1–23.8 เข้าด้วยกัน ให้ signal ที่แม่นยำและน่าเชื่อถือกว่า static β + Normal threshold

5-Step Pipeline: Kalman + t-distribution

- 1. Initialization:** OLS β บน training window (≥ 250 obs) → ได้ β_0 เริ่มต้น และ $\text{Var}(\varepsilon)$ สำหรับ R; ตั้ง Q จาก walk-forward validation
- 2. Kalman update (ทุก tick/bar):** รัน predict-update cycle → ได้ β_t และ $\varepsilon_t = y_t - \beta_t \cdot x_t$
- 3. Estimate σ_t :** คำนวณ rolling std ของ ε_t (lookback 50–100 bars) → ได้ $z_t = \varepsilon_t / \sigma_t$
- 4. t-distribution threshold:** ใช้ v ที่ประมาณจาก MLE บน training window → threshold = $t_{\{\alpha/2\}}(v)$ แทน 1.96
- 5. Entry/Exit signal:**
 - Entry: $|z_t| \geq t_{\{\alpha/2\}}(v)$ AND Kalman gain $K_t < 0.3$ (β stable — guard นี้ใช้กับ log-return inputs ที่ $x_t \approx 0.01$ – 0.05 ; สำหรับ log-price inputs ต้องปรับ threshold ตาม scale ของ x_t)
 - Exit: $|z_t| \leq 0.5$ (ε กลับ mean)
 - Stop: $|z_t| \geq 3 \times t_{\{\alpha/2\}}(v)$ (ε วิ่งไกลเกิน — ไม่ใช่ reversion)

Pipeline: Kalman + t-distribution (5 steps)



รูป 23.6 — Pipeline 5 ขั้นตอน: Kalman ให้ β_t แม่นยำ → t-distribution กำหนด threshold ที่เหมาะสม

Pseudo-code: Pipeline รวม

```
# === Initialization (ครั้งเดียวบน training data) ===
beta_hat, P = ols_fit(y_train, x_train) #  $\beta_0$ ,  $P_0$  (log-return OLS, ~1.10 for
```

```

BTC/ETH)
Q, R = tune_kalman(y_train, x_train) # walk-forward
nu = mle_t_fit(epsilon_train) # ν จาก MLE
threshold = t_dist.ppf(0.975, df=nu) # เช่น 2.74
epsilon_buf = [] # rolling buffer สำหรับ  $\sigma_t$ 

# === Live Trading Loop ===
for bar in live_stream:
    y_t, x_t = bar['log_y'], bar['log_x']

    ## ② Kalman update
    P_prior = P + Q
    innovation = y_t - beta_hat * x_t
    S = x_t**2 * P_prior + R
    K = P_prior * x_t / S
    beta_hat = beta_hat + K * innovation
    P = (1 - K * x_t) * P_prior
    epsilon_t = y_t - beta_hat * x_t

    ## ③  $\sigma_t$  estimate
    epsilon_buf.append(epsilon_t)
    if len(epsilon_buf) > 100: epsilon_buf.pop(0)
    sigma_t = std(epsilon_buf)
    z_t = epsilon_t / sigma_t

    ## ⑤ Signal
    if abs(z_t) >= threshold and K < 0.3: # Entry (K<0.3 ป้องกัน trade ระหว่าง  $\beta$ 
    updating เร็ว; สำหรับ log-return inputs  $x_t \sim 0.01-0.05$ )
    signal = 'LONG_Y' if z_t < 0 else 'SHORT_Y'
    elif abs(z_t) <= 0.5: # Exit
    signal = 'CLOSE'
    elif abs(z_t) >= 3 * threshold: # Stop
    signal = 'STOP'

```

23.10 Cross-Reference ทั้งหมด

บทนี้สร้างขึ้นบนฐานของหลายบทก่อนหน้านี้ และเพิ่มเติมให้บทที่จะใช้ต่อ:

บท	เชื่อมโยงกับ ch23
บทที่ 3 §3.9	Walk-forward calibration — ใช้สำหรับ Q/R tuning ของ Kalman
บทที่ 4 §4.2b	OLS β เป็นค่าเริ่มต้น β_0 ของ Kalman Filter
บทที่ 5 §5.6	Shapiro-Wilk: ถ้า $p \geq 0.05$ ใช้ Normal; ถ้า $p < 0.05$ ใช้ t-distribution
บทที่ 5 §5.7	Crossing rate ϕ : AR(1) coefficient เดียวกับที่ใช้ประมาณ Half-life
บทที่ 10 §10.3	Robust Z-score (MAD): ทางเลือกแทน σ -based z เมื่อ outlier มาก
บทที่ 12	Kelly criterion: ปรับ position size ตาม win rate จาก t-distribution threshold
บทที่ 18 §18.8	ε_{PCP} : ตัวอย่าง fat-tailed options residual ที่ใช้ t-distribution
บทที่ 22 §22.3	ε_{RR} : Risk Reversal residual — ตัวอย่างหลักสำหรับ MLE v ใน §23.7
บทที่ 22 §22.5	ε_{VRP} : Vol Risk Premium — ประโยชน์จาก Kalman เพราะ β_{IV} เปลี่ยนตาม term structure

โจทย์บทที่ 23

โจทย์ 23.1 — Kalman Filter 1 Step (log-returns)

BTC/ETH pair โดยใช้ daily log-returns ($r_t = \Delta \log P_t$)

ให้ค่า: $\beta_0 = 1.10$, $P_0 = 0.01$, $Q = 1 \times 10^{-5}$, $R = 0.001$

Data ใหม่: $y_1 = 0.032$ (BTC log-return = +3.2%), $x_1 = 0.028$ (ETH log-return = +2.8%)

(ก) คำนวณ $P_{1|0}$ (prior covariance)

(ข) คำนวณ S (innovation variance) และ Kalman gain K_1

(ค) คำนวณ β_1 (updated estimate), P_1 , และ ε_1

► คำตอบ

โจทย์ 23.2 — MLE ประมาณ v

ε_{RR} ของ BTC 25 Δ Risk Reversal มีสถิติต่อไปนี้จาก 252 observations:

Mean = 0.0%, Std = 2.1%, Skewness = -0.3, **Kurtosis (excess) = 5.8**

(ก) จาก kurtosis = 5.8 ประมาณ v คร่าวๆ ได้เท่าไร? (ใช้สูตร excess kurtosis = $6/(v-4)$ สำหรับ $v > 4$)

(ข) ถ้า $v \approx 5$ ให้คำนวณ two-tailed threshold ที่ $\alpha=5\%$ (ใช้ตารางใน §23.7)

(ค) เปรียบเทียบกับ Normal $z = 1.96$ — threshold ต่างกันกี่ %?

► คำตอบ

โจทย์ 23.3 — เปรียบเทียบ Normal vs t Signal

BTC/ETH pair: $\varepsilon_t = +2.4\sigma$ (คำนวณจาก Kalman β_t), $v = 4$ สำหรับ t-distribution

(ก) Normal (threshold = 1.96): เข้า trade หรือ wait?

(ข) $t(v=4)$ (threshold = 2.78): เข้า trade หรือ wait?

(ค) ถ้าหลังจากนี้ ε เพิ่มขึ้นเป็น $+3.2\sigma$ — ทั้งสองระบบตัดสินใจอย่างไร?

► คำตอบ

โจทย์ 23.4 — Full Pipeline

ให้ข้อมูล BTC/ETH: ε_{RR} series มี $v = 4$, $\sigma_{RR} = 2\%$, $\theta_{RR} = -8\%$

Observe $\varepsilon_{RR} = -14.5\%$ ($RR_{25\Delta} = -14.5\%$) พร้อมกับ Kalman gain $K_t = 0.18$

(K_t มาจาก Kalman Filter ที่ track β_{IV} ของ RR spread ตาม pipeline ใน §23.9)

(ก) คำนวณ z_t (ใช้ mean-adjusted ε)

(ข) เปรียบเทียบกับ $t(v=4)$ threshold = 2.776 — ควร enter trade ไหม?

(ค) ถ้า $K_t = 0.45$ แทน — pipeline ตัดสินใจอย่างไร?

► คำตอบ